

SECURITY ASSESSMENT REPORT

Mobile Application Penetration Test Report

Client Organisation	FitSync
Application Name	FitSync Pro
Application Version	1.0 (debug build — app-debug.apk)
Platform	Android (APK)
Engagement ID	ENG-FITSYNC-2026-001
Report Date	2026-08-20
Assessment Period	2026-07-20 – 2026-08-20
Prepared By	Karolis Marmokas
Classification	CONFIDENTIAL

Table of Contents

Table of Contents 2

Version History 4

Distribution List..... 4

Risk Summary..... 5

Key Findings Overview 5

Business Impact Statement 5

Overall Security Posture 5

In-Scope Components..... 7

Out-of-Scope Items 7

Testing Limitations & Constraints..... 7

Test Credentials & Accounts..... 7

3.1 Reconnaissance & Information Gathering 8

3.2 Static Analysis 8

3.3 Dynamic Analysis 8

3.4 API Security Testing (Endpoints Exercised by the Mobile Client)..... 8

3.5 OWASP MASVS Alignment 8

3.6 Resilience & Anti-Tampering (RASP) Assessment..... 9

Severity Rating Definitions..... 11

Remediation Priority Matrix..... 28

Overall Security Assessment..... 29

Remediation Recommendations Summary 29

Re-Test Scope 29

Appendix A – Reference Standards..... 30

Appendix B – Glossary 30

Appendix C – CVSS Scoring Notation 31

Appendix D – Disclaimer 31

Document Control

Version History

Version	Date	Author	Description
1.0	2026-08-20	Karolis Marmokas	Initial draft
1.1	2026-08-21	Karolis Marmokas	Client review amendments

Distribution List

Recipient	Role	Organisation
Karolis Marmokas	Lead Penetration Tester	FitSync

1. Executive Summary

This report presents the findings of an independent mobile application penetration test conducted on behalf of FitSync. The assessment targeted the Android application 'FitSync Pro' (version 1.0 (debug build — app-debug.apk)) and its supporting backend API infrastructure. Testing was performed between 2026-07-20 and 2026-08-20 by Karolis Marmokas, following OWASP MASVS and OWASP Mobile Top 10 guidance.

Risk Summary

The assessment identified 18 vulnerabilities across the application and API layer, spanning 2 Critical, 9 High, 7 Medium, 0 Low, and 0 Informational findings. The table below summarises the distribution of findings by severity.

CRITICAL	HIGH	MEDIUM	LOW	INFO
2	9	7	0	0

Key Findings Overview

The most significant risks identified during the assessment are summarised below:

- Two Critical-severity vulnerabilities were identified: production credentials for a third-party fitness platform (FitCloud) and an internal telemetry API hardcoded inside the APK, and a backend Firebase project whose recoverable API key would grant unauthenticated read/write access to the full member database if its security rules are left in the default open state.
- Biometric login is not cryptographically bound to a Keystore key — it can be bypassed via runtime instrumentation with no fingerprint presented at all.
- Certificate pinning is configured but its pin-set is commented out, so all HTTPS traffic, including authenticated API calls, is interceptable by any proxy with a user-installed CA certificate — no SSL-unpinning bypass is even required.
- A production authentication Bearer token is written to Android's system log (Logcat) in the release build, and a Stripe payment token is persisted in plaintext inside the local WorkManager job-queue database.

Business Impact Statement

The vulnerabilities identified in this assessment present a material risk to FitSync. Successful exploitation of the Critical and High severity findings could result in: unauthorised access to customer personally identifiable information (PII), complete compromise of user accounts, exposure of internal API credentials, and reputational and regulatory harm — including potential non-compliance with GDPR and applicable data protection legislation. Immediate remediation of Critical and High findings is strongly recommended prior to any production release or continued distribution of the application.

Overall Security Posture

Overall Rating	HIGH RISK
----------------	-----------

Rationale	Two Critical findings (hardcoded live third-party credentials shipped in the APK, and a Firebase project one open-rules mistake away from a full unauthenticated data dump), combined with nine High findings spanning authentication, cryptography, and network security, place the overall risk at HIGH. No resilience/anti-tampering controls offset this exposure.
------------------	--

2. Scope of Assessment

In-Scope Components

Component	Identifier / URL	Notes
Android Application	app-debug.apk (FitSync Pro, version 1.0)	Debug-signed APK; no R8/ProGuard obfuscation applied to this build
Backend REST API	https://api.fitsync.io (observed via intercepted client traffic)	Exercised only through the mobile client, per methodology 3.4
Third-Party Integrations	Firebase (Realtime Database, Analytics), FitCloud API, Stripe	In scope only where reachable from the mobile client

Out-of-Scope Items

- Third-party libraries and open-source components (vulnerabilities noted as informational only)
- Production infrastructure and live user data
- Social engineering and phishing attacks
- Production release-build hardening (R8/ProGuard/DexGuard obfuscation) — testing was performed against an unobfuscated debug build.

Testing Limitations & Constraints

The following limitations applied to this engagement and may affect the completeness of findings:

- Testing was performed in an isolated staging environment; behaviour in production may differ.
- No source code was provided; all static analysis was performed on the compiled APK.
- This is a self-directed practice engagement against FitSync Pro, an intentionally vulnerable training application built for this purpose — findings and remediation guidance follow the same methodology as a client engagement, but no external client sign-off applies.

Test Credentials & Accounts

Account Type	Username	Role / Permissions
Standard Member	demo_member@fitsync.io	Standard authenticated member account (self-registered test account)
Not Applicable	—	No administrative account was provided or required; admin-only data exposure was identified directly in the API response (see FINDING-006)

3. Methodology

This assessment was conducted using a black-box approach supplemented by targeted grey-box analysis of the application binary. The testing methodology aligns with the OWASP Mobile Application Security Testing Guide (MASTG), OWASP Mobile Application Security Verification Standard (MASVS) v2.0, and the OWASP API Security Top 10 (2023). Testing phases were carried out in the following order:

3.1 Reconnaissance & Information Gathering

- Examination of APK manifest (AndroidManifest.xml) for exported components, permissions, and misconfigurations.
- Identification of all third-party SDKs, libraries, and their known vulnerabilities.
- API endpoint discovery through traffic analysis and static reverse engineering.

3.2 Static Analysis

- Decompilation of the APK to review the recovered Java/Kotlin source code.
- Review of hardcoded secrets, API keys, credentials, and cryptographic implementations.
- Analysis of data storage mechanisms: SharedPreferences, SQLite databases, local files.
- Inspection of exported activities, services, broadcast receivers, and content providers.
- Certificate pinning implementation review.

3.3 Dynamic Analysis

- Runtime traffic interception through an intercepting proxy, using an Android emulator and a physical device.
- Dynamic instrumentation of the running app to bypass SSL pinning, root detection, and anti-tampering controls.
- WebView security testing: JavaScript execution, URL scheme handling, intent hijacking.
- Runtime data storage inspection: logcat output, memory inspection, clipboard monitoring.

3.4 API Security Testing (Endpoints Exercised by the Mobile Client)

API testing is performed against the endpoints the mobile client actually calls, observed and manipulated through the intercepting proxy. Assessing endpoints the client does not exercise, or testing the backend API directly, requires a separate API-engagement scope.

- Authentication and authorisation testing: JWT manipulation, token expiry, privilege escalation.
- Broken Object Level Authorisation (BOLA/IDOR) testing by manipulating object references in the client's intercepted requests.
- HTTP method tampering, mass assignment, and business logic abuse.
- Input validation: injection attacks (SQLi, NoSQLi, command injection), XSS in API responses.
- Sensitive data exposure in API responses and HTTP headers.

3.5 OWASP MASVS Alignment

MASVS Control Group	Description	Testing Performed
MASVS-STORAGE	Secure data storage and privacy	Yes

MASVS Control Group	Description	Testing Performed
MASVS-CRYPTO	Cryptography best practices	Yes
MASVS-AUTH	Authentication and session management	Yes
MASVS-NETWORK	Network communication security	Yes
MASVS-PLATFORM	Platform interaction	Yes
MASVS-CODE	Code quality and build settings	Yes
MASVS-RESILIENCE	Anti-tampering and anti-reversing controls	Yes

3.6 Resilience & Anti-Tampering (RASP) Assessment

Where the application is expected to resist reverse engineering and runtime tampering, the following resilience controls (OWASP MASVS-RESILIENCE) are assessed both statically (identifying whether the control is present) and dynamically (attempting to bypass it):

- Root & privilege-escalation detection (RESILIENCE-1): check for su binaries, root frameworks, and root-management app detection; attempt bypass via common root-hiding mechanisms.
- Developer-mode & tamper-signal exposure (RESILIENCE-1): confirm whether the app checks for enabled USB debugging (ADB) and bootloader/verified-boot state (e.g. unlocked or unverified boot) as supplementary signals distinct from full root.
- Emulator & virtual-device detection (RESILIENCE-4): run the app under an emulator and test for QEMU, sensor, and telephony fingerprinting.
- Debugger & anti-debugging (RESILIENCE-4): test ptrace/TracerPid and isDebuggerConnected-style checks.
- Dynamic-instrumentation detection (RESILIENCE-4): attach a dynamic-instrumentation framework and observe for instrumentation artefacts (open ports, /proc/maps entries, named pipes).
- Virtualization & app-cloning detection (RESILIENCE-4): run the app in a work-profile / app-cloner (dual-space, VirtualApp) to test container detection.
- Companion-threat awareness (RESILIENCE-4): check whether the app can recognise known repackaged/cloned copies of itself, or co-installed automation/macro tooling driving synthetic input, beyond generic root or instrumentation checks.
- App-integrity & anti-tampering (RESILIENCE-2): re-sign and repackage the APK to test signature verification and checksum/CRC integrity checks.
- Installer-source & sideload verification (RESILIENCE-2): confirm whether the app validates its installer package (Play Store vs sideloaded).
- Anti-static-analysis / obfuscation quality (RESILIENCE-3): evaluate the obfuscation applied (R8/ProGuard vs DexGuard), string encryption, control-flow flattening and native-code protection.
- Screen-capture, recording & casting protection (RESILIENCE-1): check FLAG_SECURE on sensitive screens, and whether an active screen-recording or casting session (e.g. MediaProjection) is detected rather than only screenshots being blocked.
- Accessibility-service abuse resistance (RESILIENCE-1): test whether the app detects or restricts third-party accessibility services capable of reading screen content or driving the UI programmatically — a distinct vector from generic overlay/tapjacking checks.
- Device integrity attestation: verify use of the Play Integrity API and/or hardware-backed key attestation for high-value flows.
- Layered-defense verification: after defeating one control (e.g. patching the root check), confirm whether a second, independent control detects the tamper — or whether a single bypass silently defeats every resilience control at once, a common maturity gap in shallow implementations.

Note on severity: resilience controls are defence-in-depth. Their absence is rated against the application's threat model — for high-value apps (financial, healthcare, DRM, enterprise) missing anti-instrumentation or integrity

controls can be a significant finding, whereas for low-risk apps the same gaps are typically Low or Informational. In this assessment, FitSync Pro implements none of the controls above: no root, debugger, emulator, or instrumentation detection was observed at any layer, so every resilience check in this report is scored as absent rather than bypassed.

4. Detailed Findings

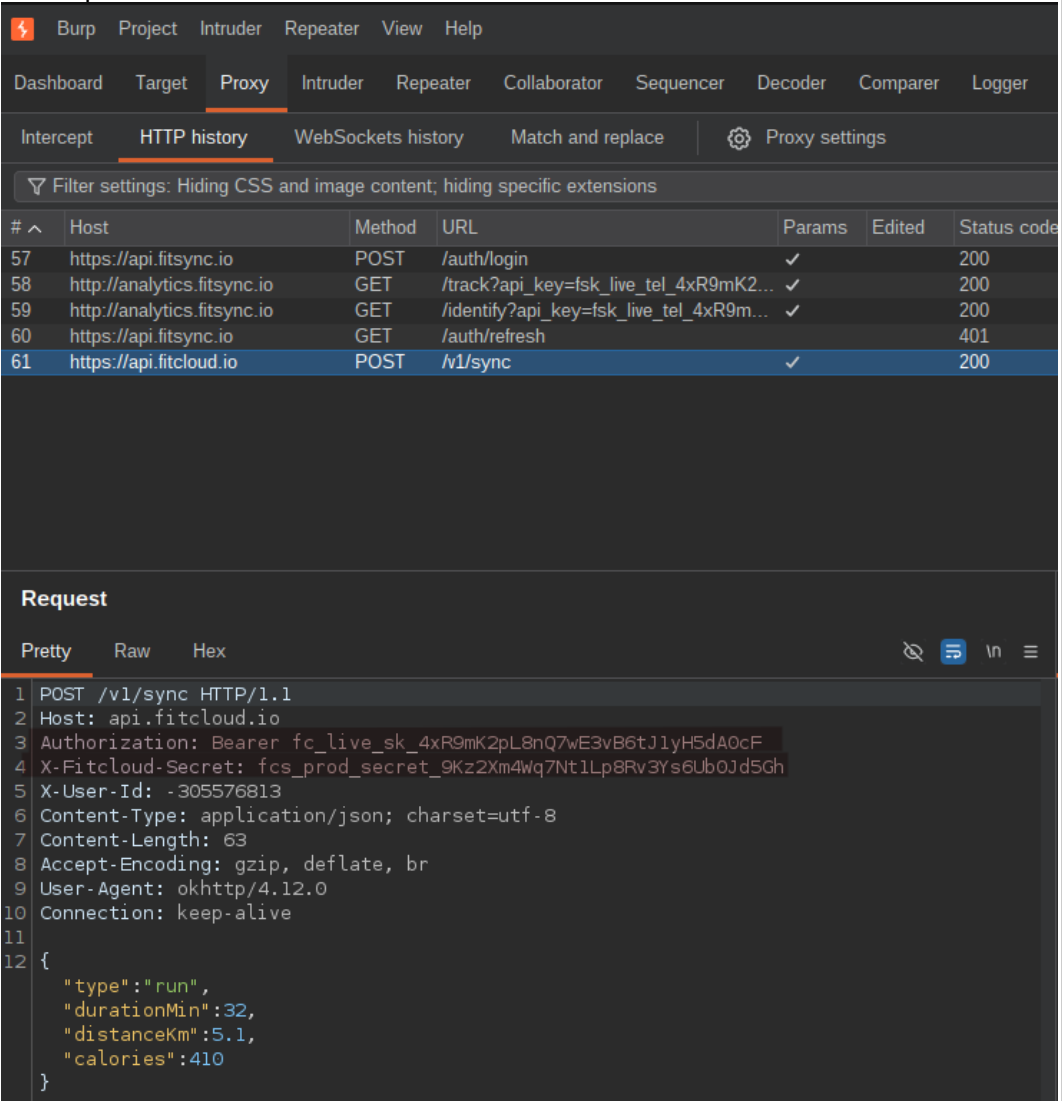
This section presents all vulnerabilities identified during the assessment. Each finding includes a severity rating derived from CVSS v3.1, a description, steps to reproduce, evidence, and remediation guidance. Findings are ordered from highest to lowest severity.

Severity Rating Definitions

Rating	CVSS Range	Description
CRITICAL	9.0 – 10.0	Immediate exploitation is likely with severe business impact. Requires urgent remediation before deployment.
HIGH	7.0 – 8.9	Significant risk with high likelihood or impact. Should be remediated as a priority within the current release cycle.
MEDIUM	4.0 – 6.9	Moderate risk. Exploitation may require specific conditions. Remediation recommended within the next planned release.
LOW	0.1 – 3.9	Minor risk with limited impact or low exploitation likelihood. Should be addressed as part of routine maintenance.
INFO	0.0	Informational observations or best-practice recommendations with no direct exploitability.

FINDING-001: Hardcoded Third-Party API Keys & Secrets in APK

Severity	CRITICAL
Vulnerability ID	V-18
Affected Component	util/AnalyticsHelper.kt, data/remote/WorkoutSyncService.kt
OWASP Category	OWASP MASVS-STORAGE-1 · Mobile Top 10 (2024): M1 Improper Credential Usage
CVSS Score	10.0 (Critical) – CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:N
Description	Four live production credentials are hardcoded in the application source and shipped inside the APK: a telemetry API key, the FitCloud live secret key, the FitCloud API secret, and the FitCloud webhook signing key. Anyone who decompiles the APK with jadx has immediate, offline access to all four.
Impact	An attacker in possession of these keys can sync arbitrary workout data, register rogue webhooks, and read the full member workout history through FitCloud. Because the credentials grant access to systems beyond the app itself, the blast radius extends to connected backend services (CVSS scope: changed).
Steps to Reproduce	1. Decompile the APK: jadx -d /tmp/fitsync app-debug.apk 2. grep -rn "fc_live\ fsk_live\ fcs_prod\ fcwh_live" /tmp/fitsync/sources/

Evidence	3. Confirm dynamically in Burp: POST https://api.fitcloud.io/v1/sync carries Authorization: Bearer fc_live_sk_... and X-FitCloud-Secret: fcs_prod_secret_... Figure V-18 — FitCloud live secret and API secret sent in request headers, recovered via Burp. 
Recommendation	Remove all third-party credentials from the mobile app. The client must call your own backend, which proxies to FitCloud using server-held keys that never touch the APK. Rotate all four exposed keys immediately — treat them as compromised. Add secret scanning (e.g. gitleaks / apk-secret-scanner) to CI to block future key commits.

FINDING-002: Firebase Realtime Database with Open Security Rules

Severity	CRITICAL
Vulnerability ID	V-06

Affected Component	Firebase Realtime Database (backend), referenced via google-services.json
OWASP Category	OWASP MASVS-STORAGE-1 · Mobile Top 10 (2024): M8 Security Misconfiguration
CVSS Score	9.1 (Critical) – CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N
Description	The Firebase project referenced by the app (project id recoverable from the embedded API key, see the related credential-exposure finding) is at risk of world-readable/writable database rules (".read": true / ".write": true). Combined with the API key shipped in the APK, unauthenticated clients could read and modify the backing datastore.
Impact	If rules are not locked, the entire member database — names, emails, phone numbers, session data — is readable without authentication, and writable data can be tampered with. This is the most severe outcome of the exposed Firebase key.
Steps to Reproduce	1. Extract the Firebase project id / API key from the APK (google-services.json or strings.xml). 2. Attempt an unauthenticated read: curl "https://<project>-default-rtdb.firebaseio.com/members.json" 3. Data returned = open rules = full unauthenticated database exposure.
Evidence	# Demo note: the demo project id does not resolve; in a real engagement # open rules return the full members/users tree. curl "https://fitsync-pro-d7e2f-default-rtdb.firebaseio.com/members.json"
Recommendation	Lock Realtime Database rules to authenticated, least-privilege access; never ship ".read": true / ".write": true. Scope reads/writes per-user via request.auth.uid; validate on the server. Restrict the Firebase API key in the Google Cloud console and rotate it.

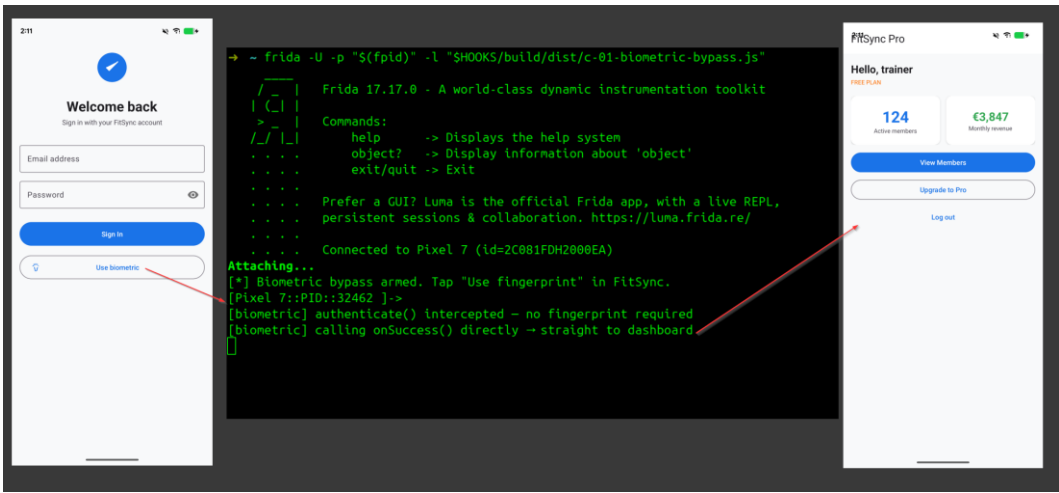
FINDING-003: Exported BroadcastReceiver Without Permission Guard

Severity	HIGH
Vulnerability ID	V-07
Affected Component	receiver/SessionReceiver.kt, AndroidManifest.xml
OWASP Category	OWASP MASVS-PLATFORM-1 · Mobile Top 10 (2024): M8 Security Misconfiguration
CVSS Score	8.0 (High) – CVSS:3.1/AV:L/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:L
Description	SessionReceiver is declared android:exported="true" with no android:permission guard, so any app on the device (or an ADB session) can send it broadcasts and invoke its privileged actions.
Impact	Any local app or ADB session can grant premium without payment, force-logout the current user (denial of service), or overwrite the session token with an attacker-controlled value via the SESSION_SYNC action.

Steps to Reproduce	1. apktool d app-debug.apk -o /tmp/fitsync_d ; grep -A3 "SessionReceiver" /tmp/fitsync_d/AndroidManifest.xml (exported="true", no permission). 2. Grant premium: adb shell am broadcast -a com.fitsync.app.GRANT_PREMIUM 3. Force logout: adb shell am broadcast -a com.fitsync.app.FORCE_LOGOUT 4. Inject token: adb shell am broadcast -a com.fitsync.app.SESSION_SYNC --es token "attacker_jwt"
Evidence	adb shell am broadcast -a com.fitsync.app.GRANT_PREMIUM adb shell am broadcast -a com.fitsync.app.FORCE_LOGOUT adb shell am broadcast -a com.fitsync.app.SESSION_SYNC --es token "fake.jwt.token"
Recommendation	Set android:exported="false" if the receiver is only used internally. If it must be exported, guard it with a signature-level android:permission and validate the caller. Never perform privilege-changing actions (premium grant, session set) from an unauthenticated broadcast.

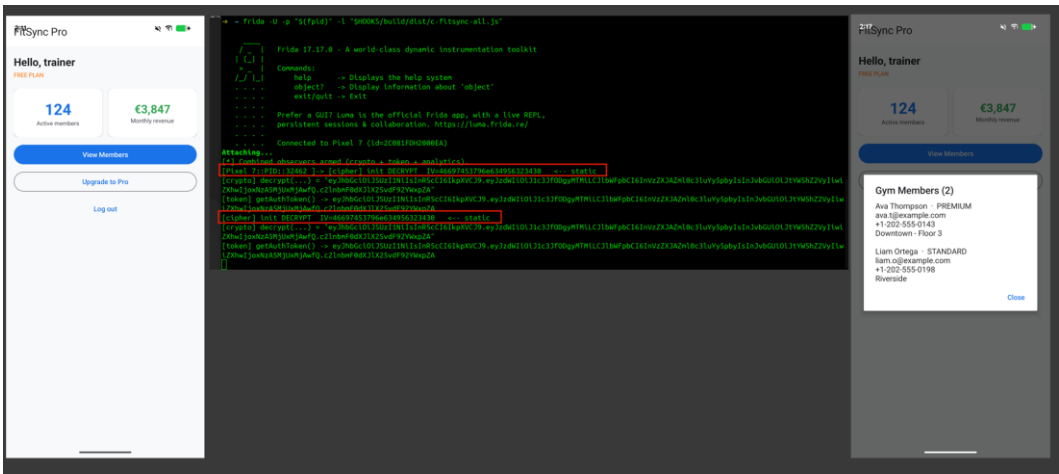
FINDING-004: Biometric Authentication Not Cryptographically Bound

Severity	HIGH
Vulnerability ID	V-01
Affected Component	util/BiometricHelper.kt — authenticate()
OWASP Category	OWASP MASVS-AUTH-2 · Mobile Top 10 (2024): M3 Insecure Authentication/Authorization
CVSS Score	7.7 (High) – CVSS:3.1/AV:L/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N
Description	The biometric prompt is invoked as prompt.authenticate(promptInfo) with no CryptoObject. Success is signalled by a plain boolean callback (onSuccess()) rather than by unlocking a Keystore-backed key, so 'authenticated' means only that a method was called — not that anything was cryptographically proven.
Impact	A Frida/Objection hook that forces the success callback bypasses login entirely, with no fingerprint presented. The biometric gate provides no real security boundary.
Steps to Reproduce	1. Confirm statically: only one argument passed to authenticate() (promptInfo, no cryptoObject). 2. Open FitSync to the login screen and attach Frida to the running process by PID. 3. Load the biometric-bypass hook, tap 'Use fingerprint', present no finger — the app lands on the dashboard, fully logged in.
Evidence	Figure V-01 — Frida hook forcing onSuccess(); the app reaches the dashboard with no fingerprint.

	
Recommendation	Generate a Keystore key with setUserAuthenticationRequired(true), wrap it in a CryptoObject, and pass it to authenticate(promptInfo, cryptoObject). In onAuthenticationSucceeded, use result.cryptoObject.cipher to decrypt a server challenge — so success requires a real biometric-unlocked key.

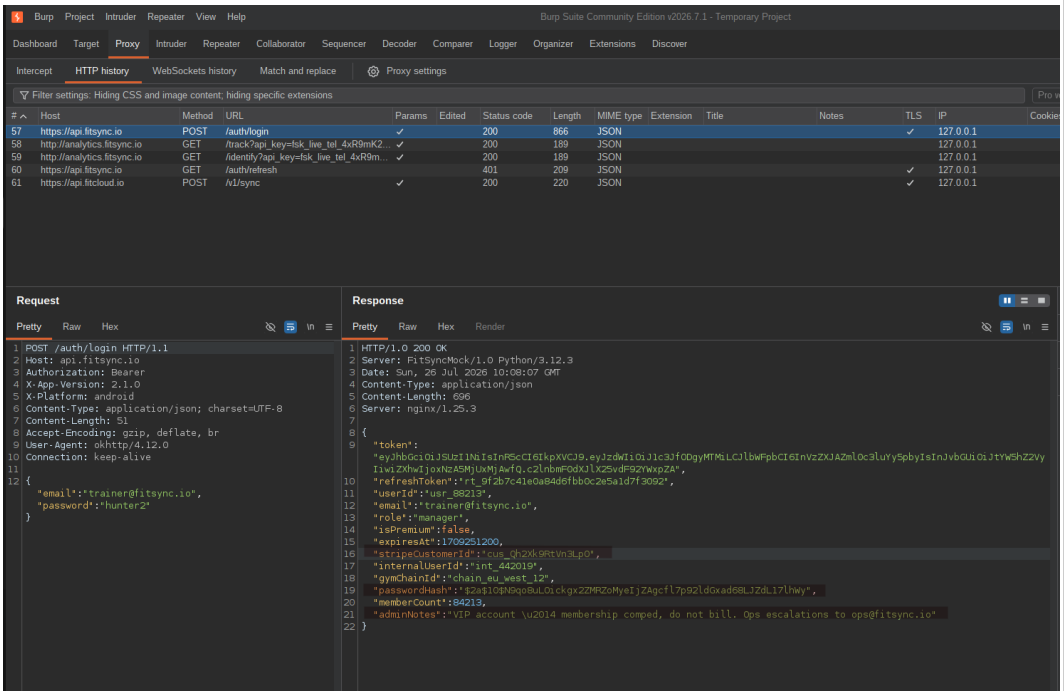
FINDING-005: AES-GCM Encryption with a Reused Static IV

Severity	HIGH
Vulnerability ID	V-02
Affected Component	util/CryptoManager.kt — STATIC_IV
OWASP Category	OWASP MASVS-CRYPTO-1 · Mobile Top 10 (2024): M10 Insufficient Cryptography
CVSS Score	7.5 (High) – CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
Description	CryptoManager encrypts sensitive fields with AES/GCM/NoPadding but reuses a single hardcoded 12-byte IV for every call. In GCM, reusing an IV with the same key is catastrophic: the XOR of two ciphertexts equals the XOR of the plaintexts, and the authentication tag becomes forgeable.
Impact	An attacker who captures two ciphertexts produced with the same key+IV can recover plaintext relationships and forge authenticated data. All locally encrypted tokens can be compromised with as few as two captures.
Steps to Reproduce	1. grep -rn "STATIC_IV val.*IVs*=" .../util/ — confirm a hardcoded IV constant. 2. Attach Frida and hook javax.crypto.Cipher.init to print the IV on every call. 3. Trigger any token read/write (e.g. View Members) and observe the IV is byte-for-byte identical every time (the bytes decode to ASCII 'FitSyncIV240').
Evidence	Figure V-02 — Two Cipher.init calls, the same IV both times — a static, reused nonce.

	
Recommendation	Generate a fresh, cryptographically random IV/nonce for every encryption and prepend it to the ciphertext. Never hardcode an IV; never reuse an IV under the same key in GCM. Consider a vetted library (e.g. Tink) that manages nonces safely.

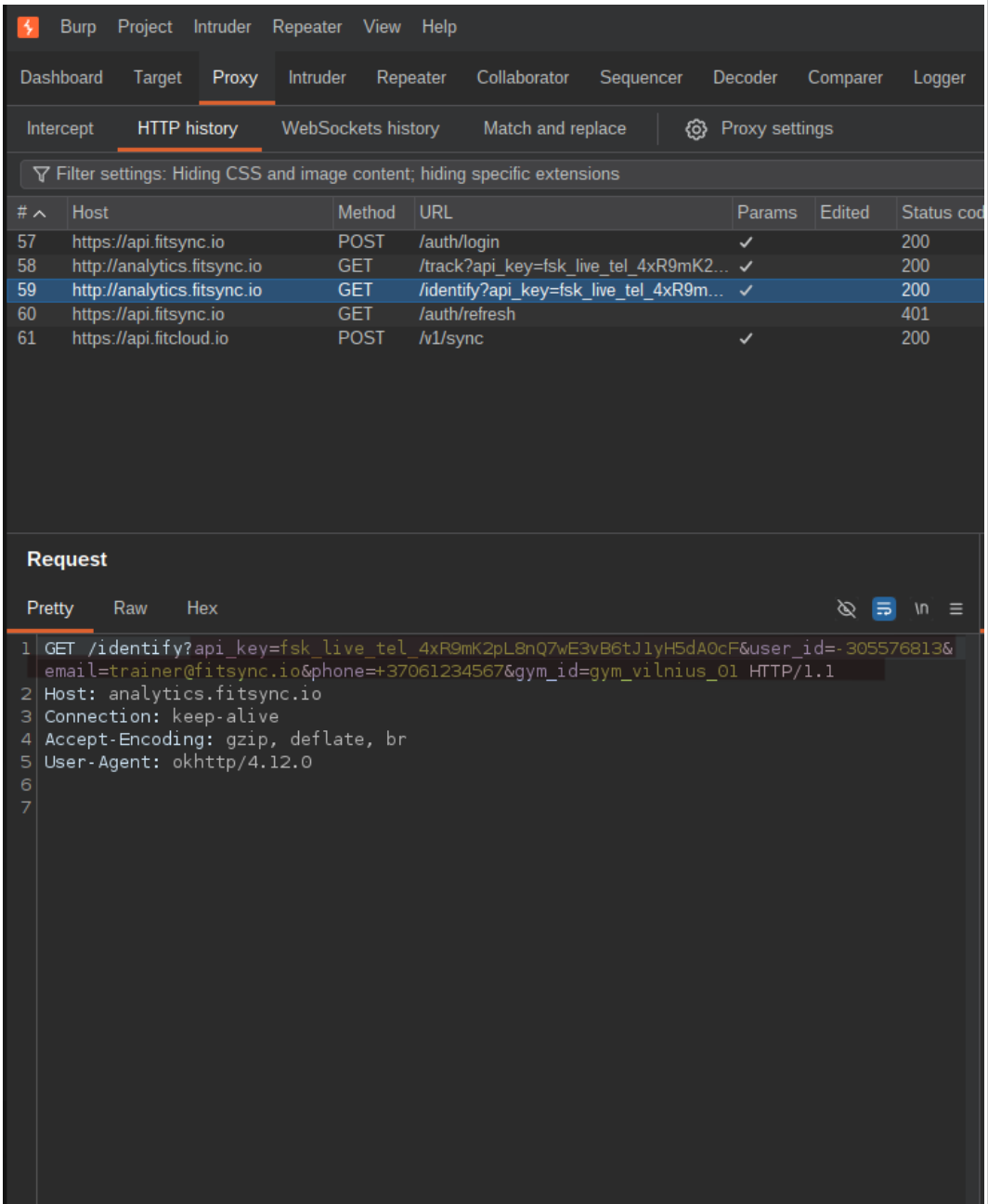
FINDING-006: Excessive Data Exposure in Login Response

Severity	HIGH
Vulnerability ID	V-16
Affected Component	data/remote/models/AuthResponse.kt — POST /auth/login
OWASP Category	OWASP API Security Top 10: API3:2023 Broken Object Property Level Authorization · Mobile M4
CVSS Score	7.5 (High) – CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
Description	The /auth/login response returns fields the mobile client never uses, including passwordHash, stripeCustomerId, memberCount, adminNotes, and internalUserId. A server returning a password hash to the client is a critical API design flaw — that value should never leave the database.
Impact	Sensitive server-side data is exposed to anyone who can observe the login response (any user of the app via a proxy). Leaked password hashes enable offline cracking; internal identifiers and admin notes aid further attacks.
Steps to Reproduce	1. Proxy the app through Burp and intercept the POST /auth/login response. 2. Inspect the JSON body for passwordHash, stripeCustomerId, adminNotes, internalUserId.
Evidence	Figure V-16 — Login response JSON exposing passwordHash and internal-only fields.

		
	Recommendation	Return a dedicated response DTO containing only fields the client needs. Apply @JsonIgnore / a response filter server-side so internal fields can never serialize to clients. Never transmit password hashes or admin metadata to the mobile client.

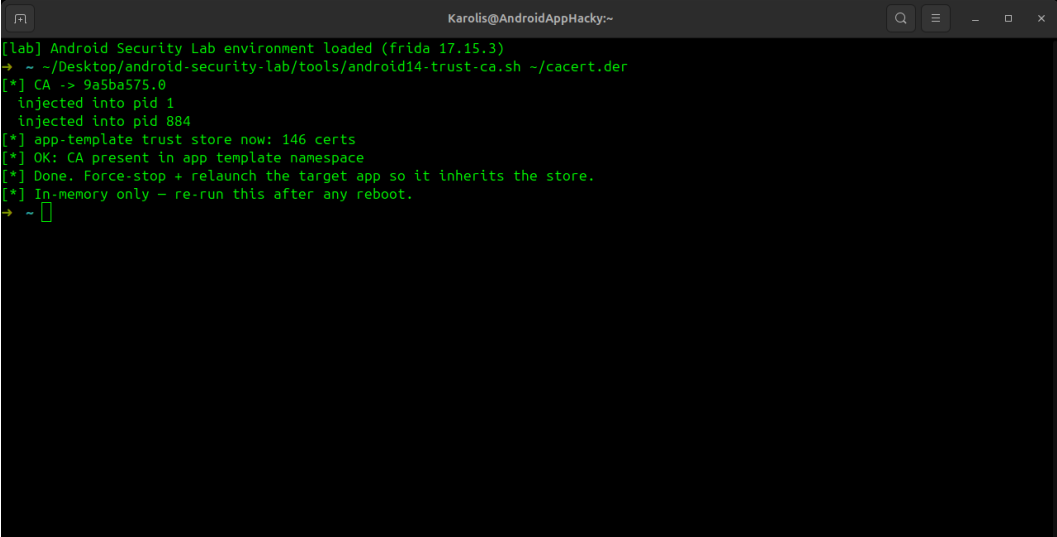
FINDING-007: Cleartext HTTP Permitted for Analytics Endpoint

Severity	HIGH
Vulnerability ID	V-17
Affected Component	res/xml/network_security_config.xml, util/AnalyticsHelper.kt
OWASP Category	OWASP MASVS-NETWORK-1 · Mobile Top 10 (2024): M5 Insecure Communication
CVSS Score	7.5 (High) – CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
Description	analytics.fitsync.io is explicitly whitelisted for cleartext traffic (cleartextTrafficPermitted="true"). All telemetry — login events, user identification, session data — is sent over plain HTTP.
Impact	Any network observer (shared Wi-Fi, malicious hotspot, on-path attacker) can read the full request, including the telemetry API key in the query string and all PII carried in the analytics payload.
Steps to Reproduce	1. apktool d app-debug.apk -o /tmp/fitsync_d ; inspect res/xml/network_security_config.xml for cleartextTrafficPermitted="true" on analytics.fitsync.io. 2. Proxy the device through Burp (no TLS setup needed for HTTP) and observe: GET http://analytics.fitsync.io/track?api_key=fsk_live_tel_...&event=login&user_id=...
Evidence	Figure V-17 — Analytics telemetry sent over plain HTTP, API key and PII in the URL.

	
Recommendation	Remove the cleartext whitelist; require HTTPS for all endpoints. Move the telemetry API key server-side; never place secrets in URL query parameters. Set cleartextTrafficPermitted="false" app-wide.

FINDING-008: Certificate Pinning Silently Disabled (Commented Out)

Severity	HIGH
Vulnerability ID	V-04
Affected Component	res/xml/network_security_config.xml
OWASP Category	OWASP MASVS-NETWORK-1 · Mobile Top 10 (2024): M5 Insecure Communication

CVSS Score	7.4 (High) – CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N
Description	The app appears to configure certificate pinning — the network security config exists, is referenced in the manifest, and sets cleartextTrafficPermitted="false" — but the actual <pin-set> definitions are commented out ("TODO: re-enable after cert rotation"). The pinning is cosmetic.
Impact	Because no pins are enforced, all HTTPS traffic is interceptable by any proxy with a user-installed CA certificate — no SSL-unpinning bypass required. This exposes every authenticated API call to a man-in-the-middle.
Steps to Reproduce	1. apktool d app-debug.apk -o /tmp/fitsync_decoded ; cat res/xml/network_security_config.xml and confirm <pin-set> is commented out. 2. Install the Burp CA certificate, set Burp as the proxy, and observe all API traffic is readable with no unpinning step.
Evidence	Figure V-04 — Traffic intercepted after a user CA install — no pin enforcement present. 
Recommendation	Enable and maintain real certificate pins in the network security config (or via OkHttp CertificatePinner). Pin to a backup key as well to survive rotation without disabling pinning. Add a test that fails the build if the pin-set is empty/commented.

FINDING-009: Deep Link Scheme Without App Links Verification (Hijackable)

Severity	HIGH
Vulnerability ID	V-09
Affected Component	AndroidManifest.xml — DeepLinkActivity, ui/auth/DeepLinkActivity.kt
OWASP Category	OWASP MASVS-PLATFORM-1 · Mobile Top 10 (2024): M8 Security Misconfiguration
CVSS Score	7.4 (High) – CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:N/A:N
Description	The fitsync:// intent-filter has no android:autoVerify="true" and there is no assetlinks.json published, so the scheme is not a verified Android App Link. A malicious app can register the same scheme.

Impact	When a password-reset email link is tapped, Android may show a chooser or silently route to the malicious app, which captures the one-time reset token — leading to account takeover.
Steps to Reproduce	1. <code>grep -A10 "DeepLinkActivity" AndroidManifest.xml</code> — confirm missing <code>autoVerify</code>. 2. Verify no Digital Asset Links: <code>curl https://fitsync.io/.well-known/assetlinks.json</code> → not found. 3. Install a second app registering <code>fitsync://</code> and observe it can receive the reset link.
Evidence	<code>curl -s "https://fitsync.io/.well-known/assetlinks.json" echo "Missing — scheme hijackable"</code>
Recommendation	Use verified Android App Links: add <code>android:autoVerify="true"</code> and publish <code>assetlinks.json</code> at the domain. Treat reset tokens as single-use and short-lived; bind them to the requesting session server-side.

FINDING-010: Stripe Payment Token Stored in Plaintext WorkManager Database

Severity	HIGH
Vulnerability ID	V-11
Affected Component	worker/PaymentSyncWorker.kt (androidx.work.workdb)
OWASP Category	OWASP MASVS-STORAGE-1 · Mobile Top 10 (2024): M9 Insecure Data Storage
CVSS Score	7.1 (High) – CVSS:3.1/AV:L/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N
Description	The payment flow enqueues a WorkManager job whose input Data includes the Stripe token as a plaintext string. WorkManager persists all input Data unencrypted in the <code>androidx.work.workdb</code> SQLite database in the app's data directory.
Impact	Any process with read access to the data directory (root, ADB backup, or another app on a compromised device) can extract Stripe tokens and card identifiers from the work database.
Steps to Reproduce	1. Initiate a payment in the app to enqueue the sync job. 2. <code>adb shell su -c "sqlite3 /data/data/com.fitsync.app/databases/androidx.work.workdb 'SELECT input FROM workspec ORDER BY last_enqueue_time DESC LIMIT 5'"</code> 3. Observe the JSON blob containing <code>stripe_token</code> in plaintext.
Evidence	<code>adb shell su -c "sqlite3 /data/data/com.fitsync.app/databases/androidx.work.workdb \ 'SELECT input FROM workspec ORDER BY last_enqueue_time DESC LIMIT 5'"</code>
Recommendation	Never place secrets in WorkManager input Data. Store a reference/opaque id and resolve the secret server-side. If local storage is unavoidable, encrypt with a Keystore-backed key and clear it after use.

FINDING-011: Auth Token Leaked to Android Logcat (Release Build)

Severity	HIGH
Vulnerability ID	V-12
Affected Component	data/remote/ApiInterceptor.kt, FitSyncApp.kt (Timber)
OWASP Category	OWASP MASVS-STORAGE-2 · Mobile Top 10 (2024): M9 Insecure Data Storage
CVSS Score	7.1 (High) – CVSS:3.1/AV:L/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N
Description	Timber is planted with a DebugTree and no release-tree configuration, and the API interceptor logs the Authorization header (Timber.e("... auth: \${request.header("Authorization")})"). The Bearer token is therefore written to logcat in production.
Impact	Any process holding READ_LOGS (granted during ADB sessions, and historically abusable) can read the Bearer token and replay it to impersonate the user against the API.
Steps to Reproduce	1. Run the app so it makes API calls. 2. adb logcat grep -i "bearer auth:" and observe the Authorization header value in the log.
Evidence	Figure V-12 — Bearer token written to logcat by the API interceptor. <pre>→ ~ adb logcat grep -i "ApiInterceptor" 07-26 13:08:07.613 12416 12524 E ApiInterceptor: API error 401 on https://api.fitsync.io/auth/refresh auth: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJ1c3JfODgyMTMLCjlbWFpbCI6InVzZXJAJAZmI0c3lu Yy5pbyIsInJvbGUiOiJtYW5hZ2VyIiwiaXNjaXhwaWJoxNzA5MjUxMjAwfQ.c2lnbmF0dXJlX25vdF92YWxpZA</pre>
Recommendation	Ship a release Timber tree that suppresses non-fatal logs or forwards to Crashlytics without headers. Never log Authorization headers or tokens. Redact sensitive values in any interceptor logging.

FINDING-012: FileProvider Exposes Entire Internal Files Directory

Severity	MEDIUM
Vulnerability ID	V-08
Affected Component	res/xml/file_paths.xml (FileProvider)
OWASP Category	OWASP MASVS-PLATFORM-1 · Mobile Top 10 (2024): M8 Security Misconfiguration
CVSS Score	5.5 (Medium) – CVSS:3.1/AV:L/AC:L/PR:N/UI:R/S:U/C:H/I:N/A:N
Description	The FileProvider path configuration declares <files-path path="." name="files"/>, exposing the whole files/ directory rather than a narrow reports/ subdirectory.
Impact	Any app that receives a content:// URI from this provider can traverse the entire files/ directory. If the app stores token backups or downloaded member CSVs there, they become readable by the receiving app.
Steps to Reproduce	1. cat res/xml/file_paths.xml and confirm path="." instead of a scoped subpath. 2. Send a content:// URI to a test app and traverse beyond the intended subdirectory.

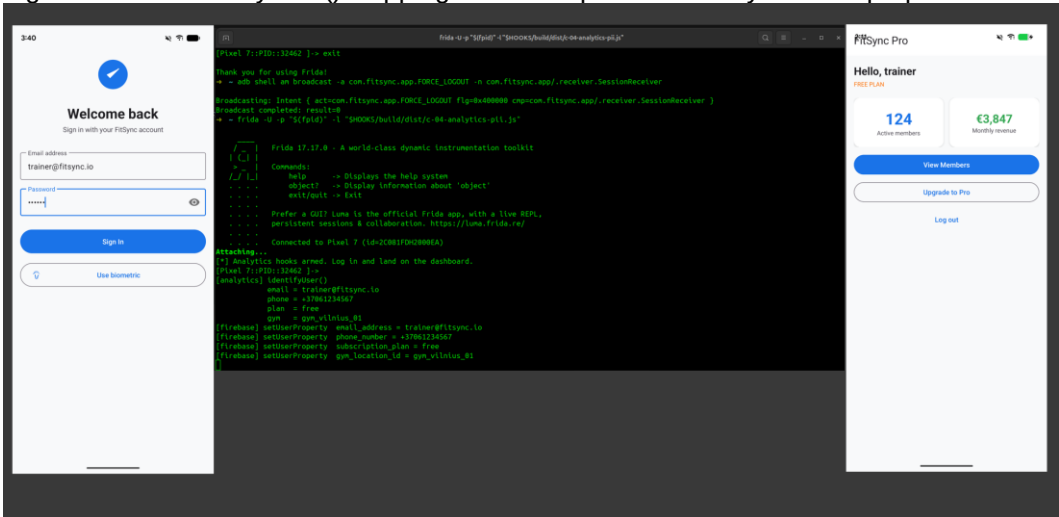
Evidence	<!-- Vulnerable: exposes all of files/ --> <files-path path="." name="files"/>
Recommendation	Scope the FileProvider to the minimum subdirectory required (e.g. path="reports/"). Grant per-URI read permission with FLAG_GRANT_READ_URI_PERMISSION and revoke after use.

FINDING-013: Firebase API Key Exposed in APK

Severity	MEDIUM
Vulnerability ID	V-05
Affected Component	google-services.json, res/values/strings.xml
OWASP Category	OWASP MASVS-STORAGE-1 · Mobile Top 10 (2024): M1 Improper Credential Usage
CVSS Score	5.3 (Medium) – CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Description	The Firebase API key is embedded in the APK (google-services.json / strings.xml) and trivially recoverable. On its own this is Medium, but it becomes Critical if the backing Firebase rules are open (see the Firebase open-rules finding).
Impact	Exposure of the project id and API key enables reconnaissance and, combined with weak backend rules, unauthenticated access to Firebase services.
Steps to Reproduce	1. apkleaks -f app-debug.apk -o leaks.txt ; grep -i firebase leaks.txt 2. Or extract google-services.json directly from the decoded APK.
Evidence	apkleaks -f app-debug.apk -o leaks.txt grep -i firebase leaks.txt # AlzaSy... key + project id
Recommendation	Restrict the API key by app package + SHA-1 and by API in the Google Cloud console. Lock Firebase security rules (see the open-rules finding) so an exposed key grants nothing. Prefer server-mediated access for sensitive data.

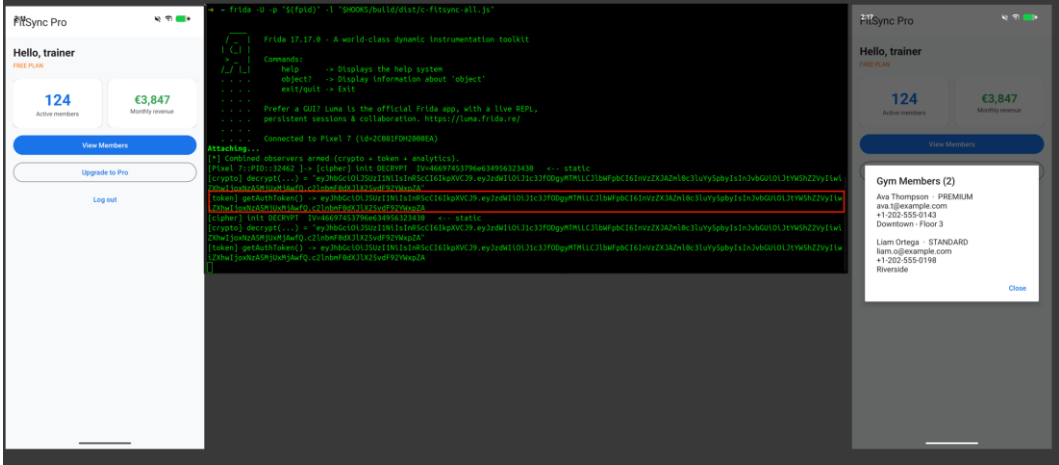
FINDING-014: PII Sent to Firebase Analytics (GDPR Violation)

Severity	MEDIUM
Vulnerability ID	V-15
Affected Component	util/AnalyticsHelper.kt — identifyUser()
OWASP Category	OWASP MASVS-PRIVACY-1 · Mobile Top 10 (2024): M6 Inadequate Privacy Controls
CVSS Score	5.3 (Medium) – CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Description	identifyUser() sets full email address and phone number as Firebase Analytics user properties (setUserProperty("email_address", email) / ("phone_number", phone)). This is PII transmitted to Google Analytics servers.

Impact	Transmitting member email and phone to analytics is personal data processing under GDPR — it requires a lawful basis/consent and constitutes a cross-border transfer to the US that needs adequate safeguards (SCCs). A regulatory and privacy exposure.
Steps to Reproduce	1. <code>grep -rn "setUserProperty\ logEvent" ../AnalyticsHelper.kt</code> and inspect the values passed. 2. Attach Frida to hook <code>FirebaseAnalytics.setUserProperty</code> and observe email/phone captured at login.
Evidence	Figure V-15 — <code>identifyUser()</code> shipping email and phone as analytics user properties. 
Recommendation	Do not send raw PII to analytics. Remove email/phone user properties. If a stable identifier is required, use a salted hash or an opaque pseudonymous id. Obtain explicit consent and document the lawful basis and any cross-border transfer.

FINDING-015: Encryption Key Derived from ANDROID_ID (Predictable Secret)

Severity	MEDIUM
Vulnerability ID	V-03
Affected Component	util/SessionManager.kt — <code>deriveLocalKey()</code>
OWASP Category	OWASP MASVS-CRYPTO-2 · Mobile Top 10 (2024): M10 Insufficient Cryptography
CVSS Score	5.1 (Medium) – CVSS:3.1/AV:L/AC:H/PR:N/UI:N/S:U/C:H/I:N/A:N
Description	The local encryption key is derived from a seed of the form <code>"fitsync_local_\${androidId}_\${Build.MODEL}"</code> . <code>ANDROID_ID</code> is a persistent but non-secret identifier, so the 'key' is predictable rather than secret, and no hardware Keystore is used.
Impact	An attacker who extracts the encrypted <code>SharedPreferences</code> file and knows the <code>ANDROID_ID</code> (readable via ADB or privileged apps) can recompute the key offline and decrypt all stored session tokens without needing the unlocked device.
Steps to Reproduce	1. <code>grep -rn "ANDROID_ID\ Settings.Secure" ../util/</code> — confirm the key is derived from <code>ANDROID_ID</code>. 2. <code>adb shell settings get secure android_id</code> ; <code>adb pull</code> the <code>fitsync_session.xml</code> prefs file. 3. Recompute <code>SHA-256("fitsync_local_<id>_<model>")</code> → AES key → decrypt the stored token.

Evidence	Figure V-03 — Session token recovered at runtime — the 'encryption' key is device-derivable. 
Recommendation	Store keys in the Android Keystore (hardware-backed); never derive keys from readable device identifiers. Bind the key to user authentication where appropriate; treat any on-device secret as recoverable by a rooted attacker.

FINDING-016: JWT Expiry (exp Claim) Not Validated Client-Side

Severity	MEDIUM
Vulnerability ID	V-13
Affected Component	data/repository/AuthRepository.kt — isTokenValid()
OWASP Category	OWASP MASVS-AUTH-3 · Mobile Top 10 (2024): M3 Insecure Authentication/Authorization
CVSS Score	4.8 (Medium) – CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:L/A:N
Description	isTokenValid() decodes the JWT payload and reads sub but never checks the exp (expiry) claim, so the client treats an expired token as valid.
Impact	A token captured elsewhere (e.g. from logcat) remains 'valid' locally after the server has expired it. The app will not trigger re-login/refresh, extending the window for replay of a stolen token.
Steps to Reproduce	1. Inspect AuthRepository.isTokenValid() and confirm no exp/expiresAt comparison. 2. Supply an expired JWT and observe the app continues to treat the session as valid locally.
Evidence	// isTokenValid() reads only 'sub' from the payload; // no check of the 'exp' claim against current time.
Recommendation	Validate the exp claim (with clock-skew tolerance) on every use; refresh or force re-login when expired. Rely on server-side token validation as the source of truth.

FINDING-017: PendingIntent with FLAG_MUTABLE (Notification Intent Hijack)

Severity	MEDIUM
----------	--------

Vulnerability ID	V-10
Affected Component	util/NotificationHelper.kt
OWASP Category	OWASP MASVS-PLATFORM-1 · Mobile Top 10 (2024): M8 Security Misconfiguration
CVSS Score	4.7 (Medium) – CVSS:3.1/AV:L/AC:H/PR:L/UI:N/S:U/C:L/I:L/A:N
Description	A notification PendingIntent is created with FLAG_MUTABLE (FLAG_UPDATE_CURRENT or FLAG_MUTABLE). Passive notification-tap intents should be FLAG_IMMUTABLE.
Impact	A privileged app or framework component can fill in or modify the intent extras (member_id, amount) before the PendingIntent fires, potentially surfacing another member's payment details via the notification deep link.
Steps to Reproduce	1. grep -rn "FLAG_MUTABLE\\PendingIntent" ../util/ and confirm FLAG_MUTABLE on a passive intent.
Evidence	PendingIntent.getActivity(ctx, 0, intent, PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_MUTABLE) // should be FLAG_IMMUTABLE
Recommendation	Use FLAG_IMMUTABLE for notification/passive PendingIntents. Only use FLAG_MUTABLE when you specifically need the receiver to fill in extras, and validate them.

FINDING-018: No FLAG_SECURE on Payment Screen (Screenshottable)

Severity	MEDIUM
Vulnerability ID	V-14
Affected Component	ui/payment/PaymentActivity.kt
OWASP Category	OWASP MASVS-PLATFORM-2 · Mobile Top 10 (2024): M8 Security Misconfiguration
CVSS Score	4.3 (Medium) – CVSS:3.1/AV:P/AC:L/PR:N/UI:R/S:U/C:H/I:N/A:N
Description	PaymentActivity.onCreate() does not set WindowManager.LayoutParams.FLAG_SECURE, so the payment screen can be captured.
Impact	Screen-recording/screenshot apps can capture the full card number, CVV, and expiry; the recent-apps thumbnail may show card details; and MediaProjection screen casts can read the screen without a visible notification.
Steps to Reproduce	1. grep -rn "FLAG_SECURE" ../ui/payment/ — empty result = flag not set. 2. Open PaymentActivity and take a screenshot / screen recording during card entry.
Evidence	// onCreate() is missing: window.setFlags(WindowManager.LayoutParams.FLAG_SECURE, WindowManager.LayoutParams.FLAG_SECURE)

Recommendation

Set FLAG_SECURE on the payment activity's window in onCreate(). Apply the same protection to any screen showing card, token, or other sensitive data.

5. Technical Risk Summary

The table below provides a consolidated overview of all findings identified during this engagement.

#	Vulnerability Title	Severity	OWASP Ref	Status	Affected Component
001	Hardcoded Third-Party API Keys & Secrets in APK	Critical	MASVS-STORAGE-1	Open	util/AnalyticsHelper.kt, data/remote/WorkoutSyncService.kt
002	Firebase Realtime Database with Open Security Rules	Critical	MASVS-STORAGE-1	Open	Firebase Realtime Database (backend)
003	Exported BroadcastReceiver Without Permission Guard	High	MASVS-PLATFORM-1	Open	receiver/SessionReceiver.kt
004	Biometric Authentication Not Cryptographically Bound	High	MASVS-AUTH-2	Open	util/BiometricHelper.kt
005	AES-GCM Encryption with a Reused Static IV	High	MASVS-CRYPTO-1	Open	util/CryptoManager.kt
006	Excessive Data Exposure in Login Response	High	API3:2023	Open	data/remote/models/AuthResponse.kt
007	Cleartext HTTP Permitted for Analytics Endpoint	High	MASVS-NETWORK-1	Open	res/xml/network_security_config.xml
008	Certificate Pinning Silently Disabled (Commented Out)	High	MASVS-NETWORK-1	Open	res/xml/network_security_config.xml
009	Deep Link Scheme Without App Links Verification (Hijackable)	High	MASVS-PLATFORM-1	Open	AndroidManifest.xml — DeepLinkActivity
010	Stripe Payment Token Stored in Plaintext WorkManager Database	High	MASVS-STORAGE-1	Open	worker/PaymentSyncWorker.kt
011	Auth Token Leaked to Android Logcat (Release Build)	High	MASVS-STORAGE-2	Open	data/remote/ApiInterceptor.kt

#	Vulnerability Title	Severity	OWASP Ref	Status	Affected Component
012	FileProvider Exposes Entire Internal Files Directory	Medium	MASVS-PLATFORM-1	Open	res/xml/file_paths.xml
013	Firebase API Key Exposed in APK	Medium	MASVS-STORAGE-1	Open	google-services.json
014	PII Sent to Firebase Analytics (GDPR Violation)	Medium	MASVS-PRIVACY-1	Open	util/AnalyticsHelper.kt
015	Encryption Key Derived from ANDROID_ID (Predictable Secret)	Medium	MASVS-CRYPTO-2	Open	util/SessionManager.kt
016	JWT Expiry (exp Claim) Not Validated Client-Side	Medium	MASVS-AUTH-3	Open	data/repository/AuthRepository.kt
017	PendingIntent with FLAG_MUTABLE (Notification Intent Hijack)	Medium	MASVS-PLATFORM-1	Open	util/NotificationHelper.kt
018	No FLAG_SECURE on Payment Screen (Screenactable)	Medium	MASVS-PLATFORM-2	Open	ui/payment/PaymentActivity.kt

Status key: Open — not yet remediated; Remediated — fixed and verified at re-test; Risk-Accepted — the client has formally accepted the residual risk.

Remediation Priority Matrix

The following remediation priorities are recommended based on risk severity and exploitability:

Priority	Severity	Action Required	Target Timeframe
P1	Critical	Immediate remediation required. Do not release to production.	Within 7 days
P2	High	Remediate prior to or during the next planned release.	Within 30 days
P3	Medium	Schedule remediation in the next development sprint.	Within 90 days
P4	Low / Info	Address during routine maintenance cycles.	Within 6 months

6. Conclusion

Overall Security Assessment

The mobile application 'FitSync Pro' and its supporting API infrastructure were assessed against the OWASP MASVS v2.0 and OWASP API Security Top 10 (2023). The assessment revealed 18 vulnerabilities, including 2 Critical and 9 High severity issues that represent significant risk to the organisation and its users.

FitSync Pro shows a consistent pattern of trusting the client boundary: secrets, cryptographic material, and privileged actions are all treated as safe once inside the APK, when in practice every one of them was recoverable through static or dynamic analysis. No anti-tampering, root-detection, or runtime-instrumentation resistance was observed at any layer, so every dynamic finding in this report was reproduced without first needing to defeat a defensive control. This reads as a pre-production application that has not yet been through a security review, rather than a previously-hardened one.

Remediation Recommendations Summary

- Immediately address all Critical findings before any production deployment or continued distribution of the application.
- Implement a secure development lifecycle (SDL) process that includes mandatory security review gates.
- Conduct developer security training focused on OWASP Mobile Top 10 and secure API design patterns.
- Implement automated static analysis tooling (SAST) as part of the CI/CD pipeline.
- Schedule a re-assessment following remediation of Critical and High severity findings to verify effective resolution.

Re-Test Scope

Upon completion of remediation activities, Karolis Marmokas recommends a targeted re-test of all Critical and High severity findings to verify that implemented fixes are effective and have not introduced additional vulnerabilities. A re-test report supplement will be issued following the re-test engagement.

7. Appendix

Appendix A – Reference Standards

This assessment was conducted in alignment with the following industry standards and frameworks:

Standard	Description	Version
OWASP MASVS	Mobile Application Security Verification Standard	v2.0 (2023)
OWASP MASTG	Mobile Application Security Testing Guide	2024
OWASP API Security Top 10	Top 10 API Security Risks	2023
OWASP Mobile Top 10	Top 10 Mobile Security Risks	2024
CVSS	Common Vulnerability Scoring System	v3.1 (v4.0 available)
CWE	Common Weakness Enumeration (MITRE)	4.x
NIST SP 800-163	Vetting the Security of Mobile Applications	Rev. 1

Appendix B – Glossary

The following terms and acronyms are used throughout this report:

Term	Definition
APK	Android Package – the distributable format for Android applications.
MASVS	Mobile Application Security Verification Standard – OWASP's framework for mobile security requirements.
IDOR	Insecure Direct Object Reference – a type of access-control flaw where object references are exposed to users without proper authorization checks.
SSL Pinning	A technique that hard-codes the expected TLS certificate or public key within a mobile application to prevent MITM attacks.
MITM	Man-in-the-Middle – an attack where a third party secretly intercepts and relays communications between two parties.
CVSS	Common Vulnerability Scoring System – a standardised framework for rating the severity of software vulnerabilities (v3.1 used throughout this report).
API	Application Programming Interface – a set of rules that allows programs to communicate with each other.
JWT	JSON Web Token – a compact, URL-safe means of representing claims between two parties, commonly used for API authentication.

Term	Definition
OWASP	Open Web Application Security Project – a nonprofit foundation that produces freely available articles, methodologies, documentation, tools, and technologies in the field of web application security.
Root Detection	A runtime check performed by an application to determine whether the device has been rooted/jailbroken, which could undermine security controls.
Static Analysis	Security testing performed without executing the application – e.g. reviewing decompiled source code, configuration files, and embedded secrets.
Dynamic Analysis	Security testing performed while the application is running – e.g. traffic interception, runtime hooks, and behavioural analysis.

Appendix C – CVSS Scoring Notation

All vulnerability severity scores in this report use the Common Vulnerability Scoring System version 3.1 (CVSS v3.1). Scores are expressed using the standard vector string notation, for example: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H (Critical, 9.8). CVSS scores are calculated using the base metric group only unless otherwise stated. The full CVSS v3.1 calculator is available at: <https://www.first.org/cvss/calculator/3.1> CVSS v4.0 is available; v3.1 is used throughout this report for consistency unless a finding states otherwise.

Appendix D – Disclaimer

This report has been prepared solely for the use of FitSync and is provided in accordance with the terms of the engagement agreement. The information contained in this report reflects the security posture of the assessed application at the time of testing only. New vulnerabilities may emerge following the completion of this assessment. Neither Karolis Marmokas nor any employees or agents acting on their behalf accept any liability for any decisions made or actions taken based on the information contained herein. Reproduction or distribution of this report, in whole or in part, without prior written consent is strictly prohibited.

END OF REPORT

FitSync — Mobile Application Penetration Test Report — 2026-08-20